

# Adding Authentication To React Apps

Easy Guide

# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>How Authentication Works</b>                                 | <b>2</b>  |
| 1.1      | What is Authentication? . . . . .                               | 2         |
| 1.2      | The Process . . . . .                                           | 2         |
| 1.3      | What is a Token? . . . . .                                      | 2         |
| 1.4      | Token Flow . . . . .                                            | 2         |
| 1.5      | Common Token Types . . . . .                                    | 3         |
| <b>2</b> | <b>Working with Query Parameters</b>                            | <b>4</b>  |
| 2.1      | What are Query Parameters? . . . . .                            | 4         |
| 2.2      | Using in React Router . . . . .                                 | 4         |
| 2.3      | Why Use Query Parameters for Auth? . . . . .                    | 4         |
| 2.4      | Common Auth Use Cases . . . . .                                 | 5         |
| 2.4.1    | Redirect After Login . . . . .                                  | 5         |
| 2.4.2    | Email Verification . . . . .                                    | 5         |
| 2.4.3    | Password Reset . . . . .                                        | 5         |
| <b>3</b> | <b>Implementing the Auth Action</b>                             | <b>6</b>  |
| 3.1      | What is an Auth Action? . . . . .                               | 6         |
| 3.2      | Creating Login Action . . . . .                                 | 6         |
| 3.3      | Using in Route . . . . .                                        | 7         |
| 3.4      | In Login Component . . . . .                                    | 7         |
| <b>4</b> | <b>Validating User Input &amp; Outputting Validation Errors</b> | <b>8</b>  |
| 4.1      | Server-Side Validation . . . . .                                | 8         |
| 4.2      | Display Errors . . . . .                                        | 9         |
| 4.3      | Highlight Invalid Fields . . . . .                              | 9         |
| 4.4      | Email Validation Rules . . . . .                                | 9         |
| <b>5</b> | <b>Adding User Login</b>                                        | <b>11</b> |
| 5.1      | Complete Login Flow . . . . .                                   | 11        |
| 5.1.1    | Step 1: Create Login Action . . . . .                           | 11        |
| 5.1.2    | Step 2: Create Login Component . . . . .                        | 11        |
| 5.1.3    | Step 3: Add Route . . . . .                                     | 12        |
| 5.1.4    | Step 4: Backend (Node/Express) . . . . .                        | 13        |

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| <b>6</b>  | <b>Attaching Auth Tokens to Outgoing Requests</b>               | <b>14</b> |
| 6.1       | Why Attach Tokens? . . . . .                                    | 14        |
| 6.2       | Adding Token to Headers . . . . .                               | 14        |
| 6.3       | Using in Components . . . . .                                   | 14        |
| 6.4       | Interceptor Pattern . . . . .                                   | 15        |
| 6.5       | Server-Side Token Validation . . . . .                          | 16        |
| <b>7</b>  | <b>Adding User Logout</b>                                       | <b>17</b> |
| 7.1       | What Logout Does . . . . .                                      | 17        |
| 7.2       | Creating Logout Action . . . . .                                | 17        |
| 7.3       | Logout Button . . . . .                                         | 17        |
| 7.4       | With Form . . . . .                                             | 18        |
| 7.5       | Server-Side Logout . . . . .                                    | 18        |
| <b>8</b>  | <b>Updating the UI Based on Auth Status</b>                     | <b>20</b> |
| 8.1       | Creating Auth Context . . . . .                                 | 20        |
| 8.2       | Using Auth in Components . . . . .                              | 21        |
| 8.3       | Show Different UI . . . . .                                     | 21        |
| 8.4       | Hide/Show Elements . . . . .                                    | 21        |
| <b>9</b>  | <b>Important: loader(s) must return null or any other value</b> | <b>23</b> |
| 9.1       | Understanding Loader Returns . . . . .                          | 23        |
| 9.2       | Why This Matters . . . . .                                      | 23        |
| 9.3       | Auth Loaders . . . . .                                          | 24        |
| 9.4       | When to Return null . . . . .                                   | 24        |
| 9.5       | Deferred Returns . . . . .                                      | 24        |
| <b>10</b> | <b>Adding Route Protection</b>                                  | <b>26</b> |
| 10.1      | Protecting Routes . . . . .                                     | 26        |
| 10.2      | Role-Based Protection . . . . .                                 | 26        |
| 10.3      | Multiple Checks . . . . .                                       | 27        |
| 10.4      | With useAuth Hook . . . . .                                     | 28        |
| <b>11</b> | <b>Adding Automatic Logout</b>                                  | <b>29</b> |
| 11.1      | Token Expiration . . . . .                                      | 29        |
| 11.2      | Checking Token Expiration . . . . .                             | 29        |
| 11.3      | Auto-Logout on API Error . . . . .                              | 29        |
| 11.4      | Periodic Token Check . . . . .                                  | 30        |
| 11.5      | Refresh Token Strategy . . . . .                                | 31        |
| <b>12</b> | <b>Managing the Token Expiration</b>                            | <b>32</b> |
| 12.1      | Token Lifecycle . . . . .                                       | 32        |
| 12.2      | Setting Token Expiration . . . . .                              | 32        |
| 12.2.1    | Server-Side . . . . .                                           | 32        |
| 12.2.2    | Client-Side Handling . . . . .                                  | 33        |
| 12.3      | Proactive Refresh . . . . .                                     | 33        |
| 12.4      | Silent Refresh . . . . .                                        | 34        |
| 12.5      | Logout Warning . . . . .                                        | 34        |

# Chapter 1

## How Authentication Works

### 1.1 What is Authentication?

Authentication is proving you are who you say you are. It's like showing your ID at a store.

### 1.2 The Process

1. User enters username/email and password
2. App sends to server (securely)
3. Server checks credentials against database
4. If correct: Server sends a token
5. If wrong: Server rejects the login

### 1.3 What is a Token?

A token is a special string that proves you're logged in. Think of it like a concert ticket:

- You show your ticket at the door
- Door person checks if it's valid
- If valid, you get in
- If expired, you don't get in

### 1.4 Token Flow

```
User logs in
↓
Server verifies password
↓
Server creates token
↓
```

Token sent to browser  
↓  
Browser stores token  
↓  
Every request includes token  
↓  
Server verifies token  
↓  
If valid: User can access  
↓  
If invalid/expired: User logged out

### 1.5 Common Token Types

- **JWT (JSON Web Token)** - Most common, self-contained
- **Session ID** - Older method, stored on server
- **OAuth Token** - For third-party services (Google, Facebook)

**Key Concept:** Tokens prove you're logged in without storing login state on server!

## Chapter 2

# Working with Query Parameters

### 2.1 What are Query Parameters?

Extra data at the end of a URL:

```
https://example.com/login?redirect=/dashboard
                        ~~~~~
                        query parameter
```

### 2.2 Using in React Router

Get query params with `useSearchParams`:

```
import { useSearchParams } from 'react-router-dom';

function Login() {
  const [searchParams] = useSearchParams();
  const redirectTo = searchParams.get('redirect');

  return (
    <div>
      <p>Will redirect to: {redirectTo || 'dashboard'}</p>
    </div>
  );
}
```

### 2.3 Why Use Query Parameters for Auth?

- Save where user wanted to go before login
- Redirect back after login completes
- Pass temporary tokens for password reset
- Share login-required links

## 2.4 Common Auth Use Cases

### 2.4.1 Redirect After Login

```
// Login URL
/login?redirect=/dashboard

// After login, send user to /dashboard
const redirectPath = searchParams.get('redirect') || '/';
navigate(redirectPath);
```

### 2.4.2 Email Verification

```
// Email verification link
/verify?token=abc123xyz&email=user@example.com

// Use token and email to verify account
const token = searchParams.get('token');
const email = searchParams.get('email');
```

### 2.4.3 Password Reset

```
// Reset link from email
/reset-password?token=xyz789

// User enters new password with this token
const resetToken = searchParams.get('token');
```

**Tip:** Always validate tokens on the server!

## Chapter 3

# Implementing the Auth Action

### 3.1 What is an Auth Action?

An action function that handles login/logout. It processes the form and manages authentication state.

### 3.2 Creating Login Action

```
export async function loginAction({ request }) {
  const formData = await request.formData();
  const email = formData.get('email');
  const password = formData.get('password');

  // Send to server
  const response = await fetch('/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  if (!response.ok) {
    return { error: 'Invalid email or password' };
  }

  const data = await response.json();

  // Store token
  localStorage.setItem('authToken', data.token);

  // Get redirect destination
  const searchParams = new URL(request.url).searchParams;
  const redirectTo = searchParams.get('redirect') || '/dashboard';

  // Redirect to saved page
  return redirect(redirectTo);
}
```



```
}
```

### 3.3 Using in Route

```
const routes = [  
  {  
    path: '/login',  
    element: <Login />,  
    action: loginAction  
  }  
];
```

### 3.4 In Login Component

```
import { Form, useActionData, useNavigation } from 'react-router-dom';  
  
function Login() {  
  const actionData = useActionData();  
  const navigation = useNavigation();  
  
  const isLoading = navigation.state === 'submitting';  
  
  return (  
    <Form method="post">  
      {actionData?.error && (  
        <p className="error">{actionData.error}</p>  
      )}  
  
      <input name="email" type="email" />  
      <input name="password" type="password" />  
  
      <button disabled={isLoading}>  
        {isLoading ? 'Logging in...' : 'Login'}  
      </button>  
    </Form>  
  );  
}
```

**Pattern:** Action handles login, stores token, redirects user!

## Chapter 4

# Validating User Input & Outputting Validation Errors

### 4.1 Server-Side Validation

```
export async function loginAction({ request }) {
  const formData = await request.formData();
  const email = formData.get('email');
  const password = formData.get('password');

  const errors = {};

  // Validate email
  if (!email) {
    errors.email = 'Email is required';
  } else if (!email.includes('@')) {
    errors.email = 'Invalid email format';
  }

  // Validate password
  if (!password) {
    errors.password = 'Password is required';
  } else if (password.length < 6) {
    errors.password = 'Password too short';
  }

  // Return errors if any
  if (Object.keys(errors).length > 0) {
    return { errors };
  }

  // Proceed with login
  // ...
}
```

## 4.2 Display Errors

```
function Login() {
  const actionData = useActionData();

  return (
    <Form method="post">
      <div>
        <input name="email" type="email" />
        {actionData?.errors?.email && (
          <p className="error">{actionData.errors.email}</p>
        )}
      </div>

      <div>
        <input name="password" type="password" />
        {actionData?.errors?.password && (
          <p className="error">{actionData.errors.password}</p>
        )}
      </div>

      <button>Login</button>
    </Form>
  );
}
```

## 4.3 Highlight Invalid Fields

```
<input
  name="email"
  className={actionData?.errors?.email ? 'error' : ''}
  aria-invalid={!!actionData?.errors?.email}
/>
```

```
CSS:
input.error {
  border: 2px solid red;
  background-color: rgba(255, 0, 0, 0.05);
}
```

## 4.4 Email Validation Rules

```
// Basic check
if (!email.includes('@')) return 'Invalid email';

// Better check
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
if (!emailRegex.test(email)) return 'Invalid email';

// Check if email already exists (DB lookup)
const existing = await User.findOne({ email });
if (existing) return 'Email already registered';
```

**Critical:** Always validate on the server!

## Chapter 5

# Adding User Login

### 5.1 Complete Login Flow

#### 5.1.1 Step 1: Create Login Action

```
import { redirect } from 'react-router-dom';

export async function loginAction({ request }) {
  const formData = await request.formData();
  const email = formData.get('email');
  const password = formData.get('password');

  const response = await fetch('/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  if (!response.ok) {
    const error = await response.json();
    return { error: error.message };
  }

  const { token, user } = await response.json();

  // Store token
  localStorage.setItem('authToken', token);
  localStorage.setItem('user', JSON.stringify(user));

  // Redirect
  return redirect('/dashboard');
}
```

#### 5.1.2 Step 2: Create Login Component

```
function Login() {
```

```
const actionData = useActionData();
const navigate = useNavigate();

return (
  <div className="login-page">
    <h1>Login</h1>

    {actionData?.error && (
      <div className="error-message">
        {actionData.error}
      </div>
    )}

    <Form method="post">
      <div>
        <label>Email:</label>
        <input name="email" type="email" required />
      </div>

      <div>
        <label>Password:</label>
        <input name="password" type="password" required />
      </div>

      <button type="submit">Login</button>
    </Form>

    <p>
      Don't have an account?
      <Link to="/signup">Sign up</Link>
    </p>
  </div>
);
}
```

### 5.1.3 Step 3: Add Route

```
const routes = [
  {
    path: '/login',
    element: <Login />,
    action: loginAction
  },
  {
    path: '/dashboard',
    element: <Dashboard />,
    loader: requireAuth
  }
]
```

```
];
```

### 5.1.4 Step 4: Backend (Node/Express)

```
app.post('/api/login', async (req, res) => {
  const { email, password } = req.body;

  // Find user
  const user = await User.findOne({ email });
  if (!user) {
    return res.status(401).json({ message: 'User not found' });
  }

  // Check password
  const isValid = await user.checkPassword(password);
  if (!isValid) {
    return res.status(401).json({ message: 'Invalid password' });
  }

  // Create token
  const token = jwt.sign({ userId: user.id }, SECRET);

  res.json({
    token,
    user: { id: user.id, email: user.email, name: user.name }
  });
});
```

**Complete Pattern:** Action + Component + Route + Backend!

## Chapter 6

# Attaching Auth Tokens to Outgoing Requests

### 6.1 Why Attach Tokens?

Server needs to verify who you are with every request.

### 6.2 Adding Token to Headers

```
async function fetchWithAuth(url, options = {}) {
  const token = localStorage.getItem('authToken');

  const headers = {
    'Content-Type': 'application/json',
    ...options.headers
  };

  // Add token if available
  if (token) {
    headers.Authorization = `Bearer ${token}`;
  }

  return fetch(url, {
    ...options,
    headers
  });
}

// Usage
const response = await fetchWithAuth('/api/user/profile');
const data = await response.json();
```

### 6.3 Using in Components

```
function UserProfile() {
```



```
const [user, setUser] = useState(null);

useEffect(() => {
  const loadUser = async () => {
    const response = await fetchWithAuth('/api/user/profile');
    if (response.ok) {
      const data = await response.json();
      setUser(data);
    }
  };

  loadUser();
}, []);

return user ? <div>{user.name}</div> : <p>Loading...</p>;
}
```

## 6.4 Interceptor Pattern

Create a wrapper for all API calls:

```
// api/client.js
export const apiClient = {
  async get(url) {
    return this.request(url, { method: 'GET' });
  },

  async post(url, body) {
    return this.request(url, {
      method: 'POST',
      body: JSON.stringify(body)
    });
  },

  async request(url, options = {}) {
    const token = localStorage.getItem('authToken');

    const headers = {
      'Content-Type': 'application/json',
      ...options.headers
    };

    if (token) {
      headers.Authorization = `Bearer ${token}`;
    }

    const response = await fetch(url, { ...options, headers });
  }
};
```

```
    if (!response.ok) {
      throw new Error(`API error: ${response.status}`);
    }

    return response.json();
  }
};

// Usage in components
const user = await apiClient.get('/api/user/profile');
```

### 6.5 Server-Side Token Validation

```
// Express middleware
function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({ error: 'No token' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, SECRET);
    req.userId = decoded.userId;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Invalid token' });
  }
}

// Use in protected routes
app.get('/api/user/profile', requireAuth, (req, res) => {
  const user = getUserById(req.userId);
  res.json(user);
});
```

**Pattern:** Every request includes Authorization header with token!

## Chapter 7

# Adding User Logout

### 7.1 What Logout Does

1. Delete token from browser
2. Redirect to login page
3. Clear any stored user data

### 7.2 Creating Logout Action

```
export async function logoutAction() {  
  // Remove token from storage  
  localStorage.removeItem('authToken');  
  localStorage.removeItem('user');  
  
  // Redirect to login  
  return redirect('/login');  
}
```

### 7.3 Logout Button

```
function LogoutButton() {  
  const navigate = useNavigate();  
  
  const handleLogout = async () => {  
    // Remove token  
    localStorage.removeItem('authToken');  
    localStorage.removeItem('user');  
  
    // Redirect  
    navigate('/login');  
  };  
  
  return <button onClick={handleLogout}>Logout</button>;  
}
```

## 7.4 With Form

```
const routes = [
  {
    path: '/logout',
    action: logoutAction
  }
];

function Header() {
  return (
    <header>
      <Form method="post" action="/logout">
        <button type="submit">Logout</button>
      </Form>
    </header>
  );
}
```

## 7.5 Server-Side Logout

Some apps track logout on server:

```
app.post('/api/logout', requireAuth, (req, res) => {
  // Optional: Add token to blacklist
  // Optional: Invalidate session

  res.json({ message: 'Logged out' });
});

// Client
export async function logoutAction() {
  const token = localStorage.getItem('authToken');

  // Notify server
  if (token) {
    await fetch('/api/logout', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });
  }

  // Remove local storage
  localStorage.removeItem('authToken');

  return redirect('/login');
```

```
}
```

**Complete:** Clear storage + notify server + redirect to login!

## Chapter 8

# Updating the UI Based on Auth Status

### 8.1 Creating Auth Context

Track logged-in state across app:

```
import { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    // Check if user is logged in
    const storedUser = localStorage.getItem('user');
    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
    setIsLoading(false);
  }, []);

  const isLoggedIn = !!user;

  return (
    <AuthContext.Provider value={{ user, isLoggedIn, isLoading }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

## 8.2 Using Auth in Components

```
function Header() {
  const { user, isLoggedIn } = useAuth();

  return (
    <header>
      {isLoggedIn ? (
        <>
          <p>Welcome, {user.name}</p>
          <button>Logout</button>
        </>
      ) : (
        <>
          <Link to="/login">Login</Link>
          <Link to="/signup">Sign Up</Link>
        </>
      )}
    </header>
  );
}
```

## 8.3 Show Different UI

```
function Dashboard() {
  const { isLoggedIn, isLoading, user } = useAuth();

  if (isLoading) return <p>Loading...</p>;

  if (!isLoggedIn) {
    return <Navigate to="/login" />;
  }

  return (
    <div>
      <h1>Dashboard for {user.name}</h1>
      { /* Protected content */ }
    </div>
  );
}
```

## 8.4 Hide/Show Elements

```
function NavBar() {
  const { isLoggedIn } = useAuth();

  return (
```

```
<nav>
  <Link to="/">Home</Link>

  {isLoggedIn && (
    <>
      <Link to="/dashboard">Dashboard</Link>
      <Link to="/profile">Profile</Link>
    </>
  )}

  {!isLoggedIn && (
    <Link to="/login">Login</Link>
  )}
</nav>
);
}
```

**Pattern:** Auth context provides state to entire app!



## Chapter 9

# Important: loader()s must return null or any other value

### 9.1 Understanding Loader Returns

Loaders must return something. Even if no data:

```
// WRONG - loader without return
export async function dashboardLoader() {
  const user = await fetchUser();
  // Missing return!
}
```

```
// CORRECT - always return
export async function dashboardLoader() {
  const user = await fetchUser();
  return user;
}
```

```
// CORRECT - return null if no data
export async function pageLoader() {
  return null;
}
```

### 9.2 Why This Matters

React Router expects a return value. Without it:

- Component doesn't render properly
- `useLoaderData()` returns undefined
- Suspense boundaries behave unexpectedly

## 9.3 Auth Loaders

Protected page loaders must return auth status:

```
export async function protectedLoader() {
  const token = localStorage.getItem('authToken');

  if (!token) {
    throw redirect('/login');
  }

  const user = await fetchUser(token);
  return user; // Always return something
}
```

## 9.4 When to Return null

```
// If loader just validates, return null
export async function authCheckLoader() {
  const token = localStorage.getItem('authToken');

  if (!token) {
    throw redirect('/login');
  }

  // Validation passed, return null
  return null;
}

// Component doesn't need data
function ProtectedPage() {
  useLoaderData(); // Returns null
  return <div>Protected content</div>;
}
```

## 9.5 Deferred Returns

With `defer()`, must return:

```
import { defer } from 'react-router-dom';

export async function dashboardLoader() {
  const user = await fetchUser();
  const analytics = fetchAnalytics();

  return defer({
    user,
    analytics
  });
}
```

```
});  
}
```

**Rule:** Loaders always return something (data, null, deferred, or redirect)!

## Chapter 10

# Adding Route Protection

### 10.1 Protecting Routes

Only allow logged-in users:

```
function requireAuth({ request }) {
  const token = localStorage.getItem('authToken');

  if (!token) {
    // Redirect to login, remember where they tried to go
    const params = new URLSearchParams();
    params.set('redirect', new URL(request.url).pathname);
    throw redirect(`~/login?${params.toString()}`);
  }

  return null;
}

// Use in route
const routes = [
  {
    path: '/dashboard',
    element: <Dashboard />,
    loader: requireAuth
  }
];
```

### 10.2 Role-Based Protection

Check user role:

```
function requireAdmin({ request }) {
  const token = localStorage.getItem('authToken');

  if (!token) {
    throw redirect('/login');
  }
}
```

```
}

const user = JSON.parse(localStorage.getItem('user'));

if (user.role !== 'admin') {
  throw redirect('/unauthorized');
}

return null;
}

const routes = [
  {
    path: '/admin',
    element: <Admin />,
    loader: requireAdmin
  }
];
```

### 10.3 Multiple Checks

```
async function requireAuth({ request }) {
  const token = localStorage.getItem('authToken');

  if (!token) {
    return redirect('/login');
  }

  // Verify token is still valid
  try {
    const response = await fetch('/api/verify-token', {
      headers: {
        'Authorization': `Bearer ${token}`
      }
    });

    if (!response.ok) {
      localStorage.removeItem('authToken');
      return redirect('/login');
    }
  } catch (error) {
    return redirect('/login');
  }

  return null;
}
```

## 10.4 With useAuth Hook

```
function ProtectedComponent() {  
  const { isLoggedIn, isLoading } = useAuth();  
  
  if (isLoading) return <Spinner />;  
  
  if (!isLoggedIn) {  
    return <Navigate to="/login" />;  
  }  
  
  return <Dashboard />;  
}
```

**Pattern:** Check token in loader, redirect if missing!

## Chapter 11

# Adding Automatic Logout

### 11.1 Token Expiration

Tokens expire for security. When expired, log user out.

### 11.2 Checking Token Expiration

```
function isTokenExpired(token) {
  try {
    // Decode JWT (JWT has 3 parts separated by dots)
    const parts = token.split('.');
    const payload = JSON.parse(atob(parts[1]));

    // Check expiration time
    const expiryTime = payload.exp * 1000; // Convert to milliseconds
    return Date.now() > expiryTime;
  } catch {
    return true; // Invalid token
  }
}

// Usage
const token = localStorage.getItem('authToken');
if (isTokenExpired(token)) {
  localStorage.removeItem('authToken');
  navigate('/login');
}
```

### 11.3 Auto-Logout on API Error

```
export const apiClient = {
  async request(url, options = {}) {
    const token = localStorage.getItem('authToken');

    const headers = {
```

```

    'Content-Type': 'application/json',
    ...options.headers
  };

  if (token) {
    headers.Authorization = `Bearer ${token}`;
  }

  const response = await fetch(url, { ...options, headers });

  // If 401 Unauthorized, token expired
  if (response.status === 401) {
    localStorage.removeItem('authToken');
    localStorage.removeItem('user');
    window.location.href = '/login';
    return;
  }

  if (!response.ok) {
    throw new Error('API error');
  }

  return response.json();
}
};

```

### 11.4 Periodic Token Check

```

export function useTokenCheck() {
  const navigate = useNavigate();

  useEffect(() => {
    // Check every 5 minutes
    const interval = setInterval(() => {
      const token = localStorage.getItem('authToken');

      if (!token || isTokenExpired(token)) {
        localStorage.removeItem('authToken');
        navigate('/login');
      }
    }, 5 * 60 * 1000);

    return () => clearInterval(interval);
  }, [navigate]);
}

// Use in main layout

```



```
function App() {  
  useTokenCheck();  
  
  return <Outlet />;  
}
```

## 11.5 Refresh Token Strategy

Keep user logged in with refresh tokens:

```
async function refreshToken() {  
  const refreshToken = localStorage.getItem('refreshToken');  
  
  if (!refreshToken) {  
    return null;  
  }  
  
  const response = await fetch('/api/refresh-token', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ refreshToken })  
  });  
  
  if (!response.ok) {  
    // Refresh failed, user must login again  
    localStorage.removeItem('authToken');  
    localStorage.removeItem('refreshToken');  
    return null;  
  }  
  
  const { token } = await response.json();  
  localStorage.setItem('authToken', token);  
  
  return token;  
}
```

**Strategy:** Check expiration, refresh tokens, auto-logout!

## Chapter 12

# Managing the Token Expiration

### 12.1 Token Lifecycle

```
Token created (user logs in)
↓
Token sent to browser (stored in localStorage)
↓
Token included in API requests
↓
Server validates token
↓
Token approaches expiration
↓
App refreshes token or redirects to login
↓
User logs out or token expires
↓
Token deleted from browser
```

### 12.2 Setting Token Expiration

#### 12.2.1 Server-Side

```
const jwt = require('jsonwebtoken');

function createToken(userId) {
  return jwt.sign(
    { userId },
    SECRET,
    { expiresIn: '1h' } // Expires in 1 hour
  );
}

// Token payload includes exp time
// {
```

```
//  userId: 123,  
//  iat: 1234567890,  
//  exp: 1234571490  
// }
```

### 12.2.2 Client-Side Handling

```
function getTokenExpiry(token) {  
  const parts = token.split('.');  
  const payload = JSON.parse(atob(parts[1]));  
  return payload.exp * 1000; // In milliseconds  
}
```

```
// How long until expiry?  
const token = localStorage.getItem('authToken');  
const expiryTime = getTokenExpiry(token);  
const timeUntilExpiry = expiryTime - Date.now();  
  
if (timeUntilExpiry < 5 * 60 * 1000) {  
  // Less than 5 minutes left, refresh soon  
}
```

## 12.3 Proactive Refresh

Refresh before expiry:

```
export function useTokenRefresh() {  
  const navigate = useNavigate();  
  
  useEffect(() => {  
    const checkAndRefresh = async () => {  
      const token = localStorage.getItem('authToken');  
  
      if (!token) return;  
  
      const expiryTime = getTokenExpiry(token);  
      const timeUntilExpiry = expiryTime - Date.now();  
  
      // Refresh when 5 minutes left  
      if (timeUntilExpiry < 5 * 60 * 1000) {  
        await refreshToken();  
      }  
    };  
  
    // Check every minute  
    const interval = setInterval(checkAndRefresh, 60 * 1000);  
  
    return () => clearInterval(interval);  
  });  
}
```

```
    }, [navigate]));
}
```

## 12.4 Silent Refresh

Refresh token without annoying user:

```
async function refreshToken() {
  const refreshToken = localStorage.getItem('refreshToken');

  try {
    const response = await fetch('/api/refresh-token', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ refreshToken })
    });

    if (response.ok) {
      const { token } = await response.json();
      localStorage.setItem('authToken', token);
      return true;
    } else {
      // Refresh failed, redirect to login
      localStorage.removeItem('authToken');
      window.location.href = '/login';
      return false;
    }
  } catch (error) {
    localStorage.removeItem('authToken');
    window.location.href = '/login';
    return false;
  }
}
```

## 12.5 Logout Warning

Warn user before expiry:

```
export function useExpiryWarning() {
  const [showWarning, setShowWarning] = useState(false);
  const [secondsLeft, setSecondsLeft] = useState(0);

  useEffect(() => {
    const checkExpiry = () => {
      const token = localStorage.getItem('authToken');
      if (!token) return;
    }
  });
}
```

```
const expiryTime = getTokenExpiry(token);
const secondsUntilExpiry = Math.floor((expiryTime - Date.now()) / 1000);

// Show warning when 2 minutes left
if (secondsUntilExpiry < 120 && secondsUntilExpiry > 0) {
  setShowWarning(true);
  setSecondsLeft(secondsUntilExpiry);
} else if (secondsUntilExpiry <= 0) {
  localStorage.removeItem('authToken');
  window.location.href = '/login';
}
};

const interval = setInterval(checkExpiry, 1000);
return () => clearInterval(interval);
}, []);

return { showWarning, secondsLeft };
}

// In component
function App() {
  const { showWarning, secondsLeft } = useExpiryWarning();

  return (
    <>
      {showWarning && (
        <div className="warning">
          Your session expires in {secondsLeft} seconds.
          <button onClick={() => refreshToken()}>
            Stay Logged In
          </button>
        </div>
      )}
    </>
  );
}
```

**Complete Strategy:** Check expiry → Refresh before expiry → Warn user → Logout!

# Quick Summary

## Authentication Flow

1. User enters credentials
2. Server validates
3. Server creates token
4. Browser stores token
5. Every request includes token
6. Server verifies token
7. User gets access or redirected

## Key Components

- **Login Action** - Handles authentication
- **Auth Context** - Shares auth state
- **Protected Routes** - Only logged-in users
- **Token Management** - Storage and refresh
- **API Client** - Attaches token to requests

## Token Management

1. Store token securely (localStorage or secure cookie)
2. Include token in every API request
3. Check token expiration regularly
4. Refresh token before expiry
5. Remove token on logout

### Best Practices

1. Always validate on the server
2. Never hardcode tokens
3. Use HTTPS for all auth requests
4. Implement token refresh
5. Show loading states during auth
6. Provide clear error messages
7. Protect sensitive routes
8. Auto-logout on token expiry
9. Warn users before logout

### Common Mistakes to Avoid

- Storing password in localStorage (store token only)
- Not validating on server
- Hardcoding tokens in code
- Not handling token expiry
- Forgetting to add token to requests
- No error handling in auth actions
- Trusting client-side validation alone

### Tools and Libraries

- JWT (JSON Web Tokens) - Token format
- localStorage - Browser storage
- useAuth Hook - State management
- React Router - Route protection
- Fetch API - Make authenticated requests

## Testing Checklist

1. Login with correct credentials
2. Login with wrong credentials
3. Protected routes redirect unauthenticated users
4. Token included in API requests
5. Logout clears token
6. Token refresh works
7. Auto-logout on expiry
8. Proper error messages
9. Form validation works

---

*Authentication is the foundation of secure React apps!*